

Machine Learning: Assignment-3

Name: Suraj Shah

Admission Number: I24AI012

Q3 Create a dictionary where the keys are multiples of 3 among the first 100 natural numbers and each value is the cube of the key.

Code:

```
cube_dict={x: x**3 for x in range(1, 101) if x%3==0}  
  
print(cube_dict)
```

Output:

```
{3: 27, 6: 216, 9: 729, 12: 1728, 15: 3375, 18: 5832, 21: 9261, 24: 13824, 27: 19683, 30: 27000, 33: 35937, 36: 46656, 39: 59319, 42: 74088, 45: 91125, 48: 110592, 51: 132651, 54: 157464, 57: 185193, 60: 216000, 63: 250047, 66: 287496, 69: 328509, 72: 373248, 75: 421875, 78: 474552, 81: 531441, 84: 592704, 87: 658503, 90: 729000, 93: 804357, 96: 884736, 99: 970299}
```

Q4 Write a code in one line to print following results:

- a. i/p: List 1=[abc] ,list2=[def]
o/p: ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']

Code:

```
list1=['abc']  
list2=['def']  
print([a+b for a in list1[0] for b in list2[0]])
```

Output:

```
• ['ad', 'ae', 'af', 'bd', 'be', 'bf', 'cd', 'ce', 'cf']
```

- b. i/p: List 1=[acdb]
o/p: {'A': 'aaa', 'C': 'ccc', 'B': 'bbb', 'D': 'ddd'}

Code:

```
list1=['acdb']  
s=list1[0]  
print({c.upper(): c*3 for c in s[:len(s)//2]+s[:len(s)//2-1:-1]})
```

Output:

```
• {'A': 'aaa', 'C': 'ccc', 'B': 'bbb', 'D': 'ddd'}
```

Q5 Write a python code to Make the following 4x7 array called nola that starts with 1 and steps by 2. However, note that the first element in each row is always 4 more than the last element in the previous row.

```
nola= [[ 1 3 5 7 9 11 13]
[17 19 21 23 25 27 29]
[33 35 37 39 41 43 45]
[49 51 53 55 57 59 61]]
```

Code:

```
import numpy as np

rows, cols=4, 7
nola=np.zeros((rows, cols), dtype=int)

num=1
for i in range(rows):
    for j in range(cols):
        nola[i][j]=num
        num+=2
    num+=4

print(nola)
```

Output:

```
[[ 1  3  5  7  9 11 13]
 [19 21 23 25 27 29 31]
 [37 39 41 43 45 47 49]
 [55 57 59 61 63 65 67]]
```

Q6 Define a roux array as a 1-D array such that, when it's reversed, it represents the sequence of square numbers 1, 4, 9, 16, ... with 0s interwoven between them.

roux array of length 5

[9 0 4 0 1]

roux array of length 8

[0 16 0 9 0 4 0 1]

Code:

```
def roux_array(n):
    arr=[]
    k=1

    while len(arr)<n:
        arr.append(k*k)
        if len(arr)<n:
            arr.append(0)
        k+=1

    return arr[::-1]

n=int(input("Enter the size of array:"))
print(roux_array(n))
```

Output:

```
Enter the size of array:5
[9, 0, 4, 0, 1]
```

Q7 Write a NumPy program to test whether each element of a 1-D array is also present in a second array.

Expected Output:

Array1: [0 10 20 40 60]

Array2: [0, 40]

Compare each element of array1 and array2

Output: [True False False True False]

Code:

```
import numpy as np

array1 = np.array(list(map(int, input("Enter elements of Array1 separated by
space: ").split()))))
array2 = np.array(list(map(int, input("Enter elements of Array2 separated by
space: ").split()))))

print("Array1:", array1)
print("Array2:", array2)

result=np.isin(array1, array2)

print("Compare each element of array1 and array2")
print("Output:", result)
```

Output:

```
Enter elements of Array1 separated by space: 0 10 20 40 60
Enter elements of Array2 separated by space: 0 40
Array1: [ 0 10 20 40 60]
Array2: [ 0 40]
Compare each element of array1 and array2
Output: [ True False False  True False]
```

Q8 Write a NumPy program to get the unique elements of an array

Expected Output:

Original array:

[10 10 20 20 30 30]

Unique elements of the above array:

[10 20 30]

Original array:

[[1 1]

[2 3]]

Unique elements of the above array:

[1 2 3]

Code:

```
import numpy as np

arr1=np.array(list(map(int, input("Enter elements of 1-D array separated by
space:").split()))
print("\nOriginal array:")
print(arr1)

unique1=np.unique(arr1)
print("Unique elements of the above array:")
print(unique1)

rows=int(input("\nEnter number of rows for 2-D array: "))
cols=int(input("\nEnter number of columns for 2-D array: "))

print("Enter elements row-wise:")
elements=[]
for i in range(rows):
    row=list(map(int, input().split()))
    elements.append(row)

arr2=np.array(elements)

print("\nOriginal array:")
print(arr2)

unique2=np.unique(arr2)
print("Unique elements of the above array:")
```

```
print(unique2)
```

Output:

```
Enter elements of 1-D array separated by space:10 10 20 20 30 30

Original array:
[10 10 20 20 30 30]
Unique elements of the above array:
[10 20 30]

Enter number of rows for 2-D array: 2

Enter number of columns for 2-D array: 2
Enter elements row-wise:
1 1
2 3

Original array:
[[1 1]
 [2 3]]
Unique elements of the above array:
[1 2 3]
```

Q9 Write a NumPy program to sort an along the first, last axis of an array.

Sample array: `[[2,5],[4,4]]`

Expected Output:

Original array:

`[[4 6]`

`[2 1]]`

Sort along the first axis:

`[[2 1]`

`[4 6]]`

Sort along the last axis:

`[[1 2]`

`[4 6]]`

Code:

```
import numpy as np

rows=int(input("Enter number of rows: "))
columns=int(input("Enter number of columns: "))

print("Enter elements row wise:")
elements=[]
for i in range(rows):
    row=list(map(int, input().split()))
    elements.append(row)

arr=np.array(elements)

print("\nOriginal array:")
print(arr)

print("\nSort along the first axis:")
arr=np.sort(arr, axis=0)
print(arr)

print("\nSort along the last axis:")
print(np.sort(arr, axis=1))
```


Output:

```
Enter number of rows: 2
Enter number of columns: 2
Enter elements row wise:
4 6
2 1

Original array:
[[4 6]
 [2 1]]

Sort along the first axis:
[[2 1]
 [4 6]]

Sort along the last axis:
[[1 2]
 [4 6]]
```

Q10 Write a NumPy program to count the occurrence of a specified item in a given NumPy array.

Sample Output:

Original array:

[10 20 20 20 20 0 20 30 30 30 0 0 20 20 0]

[4,1,7,3]

Write a NumPy program to remove nan values from a given array.

Sample Output:

Original array:

[200. 300. nan nan nan 700.]

After removing nan values:

[200. 300. 700.]

Code:

```
import numpy as np

# arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))
arr=np.array(list(map(float, input("Enter array elements (use nan for NaN) separated by space: ").split()))))

# items=list(map(int, input("Enter items to count separated by space: ").split()))

print("\nOriginal array:")
print(arr)

# counts=[np.count_nonzero(arr==i) for i in items]

# print("Counts of specified items:")
# print(counts)

clean_arr=arr[~np.isnan(arr)]

print("After removing nan values:")
print(clean_arr)
```

Output:

```
Enter array elements separated by space: 10 20 20 20 20 0 20 30 30 30 0 0 20 20 0
Enter items to count separated by space: 4 1 7 3
```

Original array:

```
[10 20 20 20 20 0 20 30 30 30 0 0 20 20 0]
```

Counts of specified items:

```
[0, 0, 0, 0]
```

```
Enter array elements (use nan for NaN) separated by space: 200. 300. nan nan nan 700.
```

Original array:

```
[200. 300. nan nan nan 700.]
```

After removing nan values:

```
[200. 300. 700.]
```

Q11 Write a NumPy program to calculate percentiles for a sequence or single-dimensional NumPy array.

Sample Output:

50th percentile (median):

3.0

40th percentile:

2.6

90th percentile:

4.6

Code:

```
import numpy as np

arr=np.array(list(map(float, input("Enter array elements separated by space: ").split()))))

print("\nArray:")
print(arr)

percentiles=list(map(float, input("Enter percentile values: ").split()))

for p in percentiles:
    print(f"{int(p)}th percentile:", np.percentile(arr, p))
```

Output:

```
Enter array elements separated by space: 10 20 20 20 20 0 20 30 30 30 0 0 20 20 0
Array:
[10. 20. 20. 20. 20.  0. 20. 30. 30. 30.  0.  0. 20. 20.  0.]
Enter percentile values: 50 40 90
50th percentile: 20.0
40th percentile: 20.0
90th percentile: 30.0
```

Q12 Write a NumPy program to find indices of elements equal to zero in a NumPy array.

Sample Output:

Original array:

[1 0 2 0 3 0 4 5 6 7 8]

Indices of elements equal to zero of the said array:

[1 3 5]

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

print("\nOriginal Array:")
print(arr)

zero_indices=np.where(arr==0)[0]

print("Indices of elements equal to zero of the said array:")
print(zero_indices)
```

Output:

```
Enter array elements separated by space: 1 0 2 0 3 0 4 5 6 7 8

Original Array:
[1 0 2 0 3 0 4 5 6 7 8]
Indices of elements equal to zero of the said array:
[1 3 5]
```

Q13 Write a NumPy program to get the indices of the sorted elements of a given array.

Expected Output:

Original array:

[1023 5202 6230 1671 1682 5241 4532]

Indices of the sorted elements of a given array:

[0 3 4 6 1 5 2]

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

print("\nOriginal Array:")
print(arr)

sorted_indices=np.argsort(arr)

print("Indices of the sorted elements of the given array:")
print(sorted_indices)
```

Output:

```
Enter array elements separated by space: 1023 5202 6230 1671 1682 5241 4532

Original Array:
[1023 5202 6230 1671 1682 5241 4532]
Indices of the sorted elements of the given array:
[0 3 4 6 1 5 2]
```

Q14 Write a NumPy program to partition a given array in a specified position and move all the smaller elements values to the left of the partition, and the remaining values to the right, in arbitrary order (based on random choice).

Sample output:

Original array:

```
[ 70 50 20 30 -11 60 50 40]
```

After partitioning on 4 the position:

```
[-11 30 20 40 50 50 60 70]
```

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

pos=int(input("Enter position partition: "))

print("\nOriginal Array:")
print(arr)

partitioned=np.partition(arr, pos)

print(f"After partitioning on position {pos}")
print(partitioned)
```

Output:

```
Enter array elements separated by space: 70 50 20 30 -11 60 50 40
Enter position partition: 3

Original Array:
[ 70  50  20  30 -11  60  50  40]
After partitioning on position 3
[-11  20  30  40  50  50  60  70]
```

Q15 Compute the euclidean distance between two arrays a and b.

Input:

[1,2,3,4,5]

[4,5,6,7,8]

Euclidean distance is: 6.7082

Code:

```
import numpy as np

a=np.array(list(map(float, input("Enter elements of first array separated by
space: ").split()))))
b=np.array(list(map(float, input("Enter elements of first array separated by
space: ").split()))))

if len(a)!=len(b):
    print("Error: Arrays must have the same length.")
else:
    distance=np.linalg.norm(a-b)
    print("Euclidean distance is:", round(distance, 4))
```

Output:

```
Enter elements of first array separated by space: 1 2 3 4 5
Enter elements of first array separated by space: 4 5 6 7 8
Euclidean distance is: 6.7082
```


Q16 Find all the peaks in a 1D numpy array a. Peaks are points surrounded by smaller values on both sides.

Input:

```
array([1, 3, 7, 1, 2, 6, 0, 1])
```

Desired Output:

```
array([2, 5])
```

where, 2 and 5 are the positions of peak values 7 and 6.

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

print("\nArray:")
print(arr)

peaks=np.where((arr[1:-1]>arr[:-2]) & (arr[1:-1]>arr[2:]))[0]+1

print("Peak positions:")
print(peaks)
```

Output:

```
Enter array elements separated by space: 1 3 7 1 2 6 0 1

Array:
[1 3 7 1 2 6 0 1]
Peak positions:
[2 5]
```

Q17 Find the index of nth repetition of number i .

Inputs

array([1, 2, 1, 1, 3, 4, 3, 1, 1, 2, 1, 1, 2])

n = 5 (from user)

i = 1 (from user)

Output

8

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

i=int(input("Enter the number to search: "))
n=int(input("Enter which occurrence to find: "))

indices=np.where(arr==i)[0]

if len(indices)<n:
    print(f"{i} does not occur {n} times in the array")
else:
    print("Index of nth repetition:", indices[n-1])
```

Output:

```
Enter array elements separated by space: 1 2 1 1 3 4 3 1 1 2 1 1 2
Enter the number to search: 1
Enter which occurrence to find: 5
Index of nth repetition: 8
```

Q18Write a NumPy program to calculate the difference between neighboring elements, element-wise, and prepend [0, 0] and append[200] to a given array.

Sample output:

Original array:

[1 3 5 7 0]

Difference between neighboring elements, element-wise, and prepend [0, 0] and

append[200] to the said array:

[0 0 2 2 2 -7 200]

Code:

```
import numpy as np

arr=np.array(list(map(int, input("Enter array elements separated by space: ").split()))))

print("\nOriginal Array:")
print(arr)

diff=np.diff(arr)

result=np.concatenate(([0, 0], diff, [200]))

print("\nDifference between neighboring elements, element-wise, and prepend [0, 0] and append [200]:")
print(result)
```

Output:

```
Enter array elements separated by space: 1 3 5 7 0

Original Array:
[1 3 5 7 0]

Difference between neighboring elements, element-wise, and prepend [0, 0] and append [200]:
[ 0  0  2  2  2 -7 200]
```

Q19 Write a NumPy program to round elements of the array to the nearest integer, and then calculate the absolute value element wise.

Sample Output:

Original array:

```
[-0.7 -1.5 -1.7 0.3 1.5 1.8 2.]
```

Round elements of the array to the nearest integer:

```
[1. 2. 2. 0. 2. 2. 2.]
```

Code:

```
import numpy as np

arr=np.array(list(map(float, input("Enter array elements separated by space: ").split()))))

print("\nOriginal Array:")
print(arr)

rounded=np.round(arr)

print("\nRounded elements of the array to the nearest integer:")
print(rounded)

abs_values=np.abs(rounded)

print("\nAbsolute values after rounding:")
print(abs_values)
```

Output:

```
Enter array elements separated by space: -0.7 -1.5 -1.7 0.3 1.5 1.8 2.

Original Array:
[-0.7 -1.5 -1.7  0.3  1.5  1.8  2. ]

Rounded elements of the array to the nearest integer:
[-1. -2. -2.  0.  2.  2.  2.]

Absolute values after rounding:
[1. 2. 2. 0. 2. 2. 2.]
```